

Business Requirements Document (BRD)

Rich Health Care — Direct Selling / MLM Platform (Simplified Scope)

Version: 1.0 (simplified) **Stack:** Fastify, TypeScript, Prisma, PostgreSQL, React/Next.js

1. Project Overview

Rich Health Care is a direct-selling (MLM) wellness company. This document defines a **lean, correct-first** scope: member registration + login, a binary-tree structure, manual payment approval, a matching income engine, a wallet/ledger, a member dashboard with a pairing diagram, and a minimal admin panel.

Everything not essential to these core flows (rank automation, extra bonus types, franchise module, CMS polish) is deliberately excluded from this phase and stubbed for later.

2. Business Model Summary

Item	Value
Joining Amount	₹999 (paid manually)
Distributor Price (DP)	₹999
MRP	₹1,499
Retail Income	Flat ₹500 margin per unit sold
Payment Method	Manual — member submits reference, Admin approves/rejects
Matching Income (gross)	₹250 per matched pair (1 Left + 1 Right)
GST Cut	5% of gross, per pair
Admin Cut	5% of gross, per pair
Matching Income (net)	₹225 per pair
Daily Matching Cap	10 pairs per member per day (₹2,250 net max/day)
Unmatched pairs	Carry forward to next day
Plan Type	Binary (1 Left leg, 1 Right leg per member)

3. User Roles

Role	Description
Guest	Views public marketing pages only

Member/Distributor	Registered under a sponsor, has wallet, tree position, can sell/refer
Admin	Approves payments, triggers matching runs, manages members/products

4. Scope

4.1 In Scope

- Public pages: Home, About, Products, Contact (minimal — no CMS bells and whistles)
- Member registration with mandatory sponsor code
- JWT authentication (member + admin roles)
- Manual payment approval workflow (joining fee + product orders)
- Binary tree placement (auto-spillover: breadth-first, left-first)
- Retail income (flat ₹500/unit, credited only on approved orders)
- Matching income engine — daily job, **10 pairs/day cap**, GST + Admin cuts, carry-forward
- Wallet + append-only ledger
- Member dashboard: wallet, transaction history, **pairing diagram (binary tree visual)**
- Admin panel: pending-payments queue, member list, manual matching trigger, product CRUD

4.2 Out of Scope (this phase)

- Payment gateway integration
 - Rank auto-qualification (rank stays a plain admin-set field, no formula)
 - Super Bonus, Fast Track Income, Team Performance Incentive, Booster Royalty
 - Franchise module
 - Achievers / Documents CMS pages
 - Automated scheduled commission runs (stays admin-triggered)
 - SMS/WhatsApp integration, multi-language, newsletter automation
-

5. Functional Requirements

5.1 Registration & Authentication

- Register with: name, phone, password, sponsor code (must be active)
- On submit: create Member as PENDING_PAYMENT, reserve binary tree slot, create ₹0 wallet
- Login: phone + password → JWT
- Password hashing via bcrypt
- Role middleware (requireRole('ADMIN'))

5.2 Manual Payment Approval

1. Member submits amount + payment reference (UTR), optional screenshot → PaymentSubmission (PENDING)
2. Admin reviews in Pending Payments queue
3. On approve:
 - Joining payment → Member becomes ACTIVE, starts counting toward upline's binary totals **from this moment only**
 - Order payment → Order becomes PAID, retail income credited

4. On reject → status REJECTED, admin note shown to member for resubmission
5. **No binary counting or wallet credit happens before approval — this is the single most important guard in the system.**

5.3 Binary Tree Placement

- Each member has one parentId and a position (LEFT/RIGHT)
- Placement rule: auto-spillover — if sponsor's direct L/R is filled, place under whichever downline member has the first open slot (breadth-first, left-first)
- Tree fetch API: return a member's subtree a few levels deep, for the dashboard diagram

5.4 Matching Income Engine

Daily job (admin-triggered), per member:

```
totalLeft = carryLeft(yesterday) + newLeftCount(today) // ACTIVE
members only
totalRight = carryRight(yesterday) + newRightCount(today) // ACTIVE
members only
```

```
pairsMatched = min(totalLeft, totalRight, 10) // HARD CAP: 10
pairs/day, non-negotiable
```

```
grossPayout = pairsMatched × ₹250
gstCut      = grossPayout × 5%
adminCut    = grossPayout × 5%
netPayout   = grossPayout - gstCut - adminCut // = ₹225/pair, max
₹2,250/day
```

```
leftoverLeft = totalLeft - pairsMatched // carried to tomorrow
leftoverRight = totalRight - pairsMatched // carried to tomorrow
```

- Only ACTIVE members (payment-approved) contribute to L/R counts
- The 10-pair cap applies **per member per day**, applied via $\min(\text{totalLeft}, \text{totalRight}, 10)$ — never let a member match more than 10 pairs in a single run regardless of tree volume
- Store a daily snapshot (BinaryVolume) per member for audit
- Ledger entry stores gross, GST cut, admin cut, and net amount for transparency

5.5 Wallet & Ledger

- Every income event writes an append-only LedgerEntry — balance is never mutated directly
- Wallet balance = sum of ledger entries
- Member-facing transaction history, filterable by type/date

5.6 Member Dashboard

- Wallet balance + transaction history
- **Pairing diagram:** a binary-tree visual (self, direct Left/Right, a couple of levels down) with today's matched-pairs count and left/right carry-forward shown alongside it. One diagram, shown on every member's dashboard.
- Current rank (static, admin-set)
- Order history

5.7 Admin Panel

- Pending Payments queue (primary daily screen) — approve/reject

- with note
 - Approved/rejected payment history (exportable CSV)
 - Member list — search, status, block/unblock
 - Manual trigger: run daily matching calculation
 - Product CRUD
 - Payout/ledger export (CSV)
 - Plan config screen (pair value, **daily cap**, joining amount, GST %, admin cut %) — not hardcoded
-

6. Simplified Data Model

```

model Member {
  id          String          @id @default(cuid())
  name        String
  phone       String          @unique
  password    String
  memberCode  String          @unique
  sponsorId   String?
  sponsor     Member?         @relation("Sponsor", fields:
[sponsorId], references: [id])
  referrals   Member[]        @relation("Sponsor")
  parentId    String?
  parent      Member?         @relation("BinaryTree", fields:
[parentId], references: [id])
  children    Member[]        @relation("BinaryTree")
  position    Position?
  role        Role            @default(MEMBER)
  rank        String?         // plain field, no auto-qualification
  logic
    status     MemberStatus @default(PENDING_PAYMENT)
    activatedAt DateTime?
    wallet     Wallet?
    volumes    BinaryVolume[]
    orders     Order[]
    payments   PaymentSubmission[]
    createdAt  DateTime      @default(now())
}

enum Position { LEFT RIGHT }
enum Role { MEMBER ADMIN }
enum MemberStatus { PENDING_PAYMENT ACTIVE BLOCKED }

model PaymentSubmission {
  id          String          @id @default(cuid())
  memberId    String
  member      Member          @relation(fields: [memberId],
references: [id])
  purpose     PaymentPurpose
  orderId     String?
  amount      Int
  referenceNo  String
  screenshotUrl String?
  status      PaymentStatus   @default(PENDING)
  adminNote   String?
  reviewedBy  String?
  reviewedAt  DateTime?
  createdAt   DateTime        @default(now())
}

enum PaymentPurpose { JOINING ORDER }
enum PaymentStatus { PENDING APPROVED REJECTED }

model Wallet {
  id          String          @id @default(cuid())

```

```

        memberId String          @unique
        member  Member          @relation(fields: [memberId], references:
[id])
        balance Int             @default(0)
        ledger  LedgerEntry[]
    }

    model LedgerEntry {
        id          String          @id @default(cuid())
        walletId    String
        wallet      Wallet          @relation(fields: [walletId], references:
[id])
        type        LedgerType
        amount      Int             // net amount credited
        grossAmount Int?
        gstCut      Int?
        adminCut    Int?
        note        String?
        createdAt   DateTime        @default(now())
    }

    enum LedgerType { RETAIL_INCOME MATCHING_INCOME JOINING_CREDIT }

    model BinaryVolume {
        id          String          @id @default(cuid())
        memberId    String
        member      Member          @relation(fields: [memberId], references:
[id])
        date        DateTime @db.Date
        leftCount   Int            @default(0)
        rightCount  Int            @default(0)
        carryLeft   Int            @default(0)
        carryRight  Int            @default(0)
        pairsMatched Int           @default(0) // hard-capped at 10 in code
        payout      Int            @default(0)
        @@unique([memberId, date])
    }

    model Product {
        id          String          @id @default(cuid())
        name        String
        dp          Int
        mrp         Int
        stock       Int            @default(0)
        active      Boolean @default(true)
        imageUrl    String?
        orders      Order[]
    }

    model Order {
        id          String          @id @default(cuid())
        memberId    String
        member      Member          @relation(fields: [memberId], references:
[id])
        productId   String
        product     Product          @relation(fields: [productId], references:
[id])
        quantity    Int
        status      OrderStatus @default(PENDING)
        createdAt   DateTime        @default(now())
    }

    enum OrderStatus { PENDING PAID CANCELLED }

```

7. API Endpoint Map

Method	Route	Purpose
POST	/auth/register	Register with sponsor code
POST	/auth/login	Login, issue JWT
GET	/member/me	Current member profile
GET	/member/tree	Subtree for pairing diagram
GET	/member/wallet	Wallet balance + ledger
POST	/orders	Create order (PENDING until approved)
POST	/payments/submit	Submit payment reference
GET	/member/payments	Own payment history
GET	/products	Public product catalog
GET	/admin/payments/pending	Pending payments queue
PATCH	/admin/payments/:id/approve	Approve payment
PATCH	/admin/payments/:id/reject	Reject payment
POST	/admin/run-matching	Trigger matching job (10-pair cap enforced)
GET	/admin/members	Member list/search
PATCH	/admin/members/:id/status	Block/unblock
GET	/admin/payouts/export	CSV export
CRUD	/admin/products	Manage products

8. Non-Functional Requirements

- All monetary values stored as integers (whole rupees) — no floats
- Every commission credit is a traceable, append-only ledger entry
- Binary counting and commission calculations must check status = ACTIVE before including a member
- **The 10-pairs/day cap is enforced in the matching job itself (min(..., 10)), not just documented** — this is a hard business rule, not a suggestion
- Plan values (pair value, daily cap, joining fee, GST %, admin cut %) live in a config table, not hardcoded

9. Build Order

1. Prisma schema + migrations
2. Auth (register/login, JWT, role middleware)
3. Manual payment approval workflow (payment submission form + admin approve/reject)
4. Binary tree placement + tree traversal endpoint
5. Wallet + ledger, retail income
6. Matching income job — **10-pair daily cap**, GST + admin cut,

- carry-forward
- 7. Member dashboard (wallet, history, pairing diagram)
- 8. Admin panel (pending payments queue, members, products, matching trigger)
- 9. Public marketing pages

End of document.